

DOSSIER DE CONCEPTION

Document de travail — à valider avant développement

Exportation, sauvegarde et restauration

Ce qui est déjà sur le téléphone n'a pas à repasser par le réseau

Archive locale, sauvegarde chiffrée sur Drive, migration d'appareil



Périmètre Talky (Flutter, Android) ■ Alanya-Backend

Statut Conception — aucune ligne de code écrite

Point d'entrée Profil → Mes médias

Dépend de Partitions de médias (rétention serveur, 410 MEDIA_EXPIRED)

Plateforme Android d'abord — iOS et iCloud hors périmètre

Date 31 août 2026

Avant-propos

Ce dossier répond à une note manuscrite posée en marge de la maquette de l'écran *Mes médias* :

Nous devons travailler à s'assurer que les données sont bien stockées sur le téléphone des inscrits à la plateforme. Cette exportation doit exploiter cet aspect, ceci nous permettra d'économiser notre bande passante mais aussi les data de l'inscrit. La seule situation où il y a une réelle exportation, c'est lorsqu'il ouvre son compte sur un nouveau téléphone ; dans ce cas les données stockées sur le drive de Google sont transférées sur le nouveau téléphone.

Deux exigences s'y lisent, et elles ne se recouvrent pas. La première est une règle d'économie : une exportation ne doit rien retélécharger de ce que le téléphone possède déjà. La seconde est un scénario : la migration vers un nouvel appareil, où un transfert réel a lieu, depuis Google Drive.

Comment lire ce document. La **partie I** décrit ce que l'inscrit voit et ce qu'on lui promet. Elle se valide sans lire la suite. La **partie II** décrit comment on le construit, fichier par fichier. Le chapitre 9 récapitule toutes les décisions arrêtées et sert de référence rapide.

Table des matières

Avant-propos	1
I Ce que l'inscrit voit	4
1 Le problème posé	5
1.1 Deux horloges qui ne tournent pas à la même vitesse	5
1.2 Ce que la note demande	5
1.3 La réponse retenue : deux mécanismes, jamais confondus	5
2 Exporter une période	7
2.1 Le parcours	7
2.2 Ce que contient l'archive	7
2.3 Les éléments manquants	7
2.4 Le dépôt sur Drive	8
3 Sauvegarder et restaurer	9
3.1 Ce qui est sauvegardé	9
3.2 Où, et sous quelle forme	11
3.3 Deux sauvegardes, jamais une seule	11
3.4 Quand elle se déclenche	11
3.5 Quand elle échoue	11
3.6 L'inscrit sans compte Google	11
3.7 La restauration sur un nouveau téléphone	12
3.8 Suppression du compte	12
II Comment on le construit	13
4 Le format d'archive	14
4.1 Le manifeste	14
4.2 L'assemblage, et pourquoi il ne compresse pas	15
4.3 Reprise après interruption	15
4.4 Le service de premier plan et sa limite de six heures	15
4.5 Mesurer avant d'annoncer	16
5 La sauvegarde	17

5.1	Fabriquer l’instantané	17
5.2	Le tri : ce qui monte, ce qui reste	17
5.2.1	Les colonnes à neutraliser	17
5.2.2	Les préférences : une liste blanche, jamais une liste noire	18
5.2.3	Le garde-fou : un test qui refuse les clés non classées	19
5.3	La clé, et son numéro de version	20
5.3.1	Le jour où le secret doit tourner	20
5.3.2	Deux points d’accès, une table qu’on n’efface jamais	20
5.3.3	Deux compléments à poser en même temps	20
5.4	Rattachement : un Drive par compte Alanya	21
5.5	Restaurer une base plus ancienne que l’application	21
6	La restauration	22
6.1	Le vrai risque n’est pas celui qu’on croit	22
6.2	Deux bonnes nouvelles dans le code existant	22
6.3	Quand demande-t-on le compte Google ?	22
6.4	La solution : un ordre, pas un verrou	23
6.5	L’état de restauration, et pourquoi il est indispensable	23
6.6	Les cinq répétitions à écrire avant le code	24
6.6.1	Ces répétitions imposent une contrainte d’architecture	24
7	Ce qui reste à lever	26
7.1	L’inconnue qui conditionne l’architecture	26
7.1.1	Isoler d’abord	26
7.1.2	Tester ensuite — et deux variantes, jamais une seule	27
7.2	Les points désormais tranchés	27
8	Impact sur le code	28
8.1	Ce qui existe et se réutilise	28
8.2	Ce qu’il faut ajouter	28
8.3	Fichiers à créer	28
8.4	Côté serveur	29
9	Décisions arrêtées	30
9.1	Hors périmètre de ce lot	31
9.2	Dépendances : état réel	31

PARTIE I

Ce que l'inscrit voit

CHAPITRE 1

Le problème posé

1.1 Deux horloges qui ne tournent pas à la même vitesse

Depuis la mise en place du stockage partitionné, Alanya tient deux règles de conservation distinctes, et c'est de leur écart que naît tout ce document.

Où	Durée	Pourquoi
Sur nos serveurs	30 jours	Le coût de stockage est le nôtre. Les partitions journalières font tomber la plus ancienne chaque nuit.
Sur le téléphone	sans limite	Le stockage appartient à l'inscrit. Tout média téléchargé reste jusqu'à désinstallation ou changement de compte.

Constat vérifié dans le code. La valeur de rétention n'est pas codée en dur dans l'application. `MediaExpiryPolicy` l'apprend du champ `retentionDays` porté par chaque réponse 410 `Gone`, et la mémorise. Elle vaut 365 jours pendant la mise en service du stockage partitionné, 30 ensuite. Toute la présente conception s'appuie sur cette valeur apprise, jamais sur une constante.

Passé la fenêtre serveur, un média n'existe plus qu'en un seul exemplaire au monde : celui posé sur le téléphone de l'inscrit. Il n'y a plus de filet. Une casse, un vol, un formatage, un changement de téléphone, et la photo est perdue — définitivement, sans que personne chez Alanya ne puisse rien y faire.

1.2 Ce que la note demande

L'exigence d'économie n'est pas un détail d'optimisation, c'est une contrainte de conception. Un inscrit qui exporte les 47 médias de mars 2026 les possède déjà tous sur son téléphone. Les redemander au serveur reviendrait à payer deux fois : notre bande passante sortante, et le forfait data de l'inscrit, pour des octets qui sont déjà à trente centimètres de l'écran.

Constat vérifié dans le code. `ChatDao.watchLocalMedia` ne liste que les messages dont `localMediaPath` est renseigné, c'est-à-dire les fichiers réellement présents sur le disque. L'écran *Mes médias* est donc déjà, par construction, une vue de ce que le téléphone possède — et fonctionne hors connexion. L'exportation hérite gratuitement de cette propriété.

1.3 La réponse retenue : deux mécanismes, jamais confondus

Un même bouton ne peut pas servir deux besoins aussi différents que « emporter mes photos de mars ailleurs » et « retrouver mon compte sur mon nouveau téléphone ». Les confondre mène à la pire des situations : deux mécanismes qui écrivent au même endroit, et plus personne ne sait lequel détient ses données le jour où il en a besoin.

	Sauvegarde	Exportation
Déclenchement	automatique, selon la fréquence choisie	l'inscrit appuie sur un bouton
Contenu	la base triée, plus les préférences retenues	les médias d'une période
Format	interne, chiffré, illisible hors d'Alanya	ouvert, en clair, lisible partout
Destination	dossier Alanya du Drive de l'inscrit	partage système, Téléchargements, ou Drive
Sert à	retrouver son compte sur un nouveau téléphone	sortir ses souvenirs d'Alanya
Coût réseau	quelques Mo	zéro, sauf récupération explicitement acceptée

Conséquence assumée. La sauvegarde ne contient pas les médias. Sur un nouveau téléphone, l'inscrit retrouve ses conversations, mais les médias de plus de 30 jours ne reviennent pas : ils n'étaient ni sur le serveur, ni dans la sauvegarde. Ce choix évite de déverser plusieurs gigaoctets dans un quota Google de 15 Go, mais il déplace l'exportation : elle n'est plus un confort, elle est **le seul filet** pour les médias anciens. C'est pourquoi l'archive exportée est elle-même relisible par la restauration (chapitre 3.7).

CHAPITRE 2

Exporter une période

2.1 Le parcours

L'inscrit arrive sur *Mes médias*, resserre par discussion, par période, par famille de média, puis appuie sur **Exporter cette période**. Une feuille s'ouvre et annonce trois choses avant tout engagement : combien d'éléments, quel poids, et ce qui manque.

1. **Récapitulatif**. « 47 éléments • 128 Mo • mars 2026 • Ama Nguema ». Le périmètre exporté est exactement celui que la grille affiche : aucune surprise, ni en plus ni en moins.
2. **Éléments manquants**, s'il y en a (voir §2.3).
3. **Destination**. Partager • Enregistrer dans Téléchargements • Déposer sur mon Drive.
4. **Assemblage** en tâche de fond, progression en notification.
5. **Remise** de l'archive selon la destination choisie.

2.2 Ce que contient l'archive

Des fichiers ouverts, rangés comme l'inscrit s'y attend, plus deux fichiers de contexte.

```
1 alanya-export-241030112-2026-03-01_2026-03-31.zip
2   index.html                                <- planche-contact, ouvrable sans
   Alanya
3   manifest.json                            <- contexte lisible par machine
4   Alanya Images/
5     2026-03-14_ama-nguema_0001.jpg
6   Alanya Videos/
7   Alanya Audio/
8   Alanya Documents/
```

Constat vérifié dans le code. L'arborescence reprend celle que `AlanyaMediaExportService` pose déjà dans `Download/Alanya/` : `Alanya Images`, `Alanya Videos`, `Alanya Documents`. Un inscrit qui a déjà exporté des médias un par un retrouve la même organisation dans le zip. Seul `Alanya Audio` est nouveau.

Le `manifest.json` est ce qui distingue une archive utile d'un tas de fichiers. Un an plus tard, un nom de fichier ne dit ni qui a envoyé la photo, ni dans quelle conversation, ni quand. Le manifeste porte tout cela — et, accessoirement, les identifiants dont la restauration a besoin pour remettre chaque fichier en face de son message.

2.3 Les éléments manquants

Un média peut manquer à l'appel pour deux raisons qui n'ont rien à voir, et l'écran doit les distinguer : promettre un téléchargement qui échouera serait pire que de ne rien promettre.

Cause	Situation	Traitement
Jamais téléchargé	Reçu, mais le téléchargement automatique était coupé, ou il a échoué. Encore présent sur le serveur.	Récupérable
Purgé du serveur	Plus vieux que la rétention. Le serveur répond 410.	Irrécupérable

La feuille d'export annonce donc, le cas échéant : « *5 éléments manquants. 3 peuvent être récupérés (40 Mo). 2 ne sont plus disponibles.* » avec une case à cocher pour le téléchargement.

Pourquoi cette entorse à la règle d'économie. Récupérer avant d'archiver consomme la bande passante que la note veut économiser. L'entorse est acceptée parce qu'elle est **explicite, chiffrée d'avance et non cochée par défaut** : l'inscrit décide, en connaissant le coût. Ce qui est proscrit, c'est le téléchargement silencieux — pas le téléchargement choisi.

Les irrécupérables ne disparaissent pas pour autant : ils sont inscrits dans le manifeste avec leur motif. Une archive incomplète qui le dit vaut infiniment mieux qu'une archive incomplète qui se tait : l'inscrit ne découvrirait le trou que le jour où il chercherait la photo.

2.4 Le dépôt sur Drive

Déposer l'archive sur le Drive personnel de l'inscrit règle le problème ouvert par la sauvegarde sans médias : ses fichiers anciens survivent à son téléphone, sans qu'Alanya porte le coût d'une sauvegarde de plusieurs gigaoctets. L'archive va dans le dossier visible **Alanya/**, celui-là même qui accueille les sauvegardes, et la restauration sait la relire.

À dire clairement dans l'écran. Cette archive est **en clair**. C'est sa raison d'être : elle doit s'ouvrir sans Alanya, sur n'importe quel ordinateur, dans dix ans. Mais cela signifie que ces photos sont lisibles dans le Drive de l'inscrit, par Google et par toute application à laquelle il a donné accès à son Drive. L'écran doit le dire avant le dépôt, pas dans les conditions d'utilisation.

CHAPITRE 3

Sauvegarder et restaurer

3.1 Ce qui est sauvegardé

La base locale `talky_chat.sqlite`, moins ce qui n'a aucun sens sur un autre téléphone, plus une poignée de préférences qui n'y sont pas. Cette formulation est plus lourde que « la base et rien d'autre », mais elle est exacte, et la différence porte sur des choses que l'inscrit remarquerait.

Une objection à traiter d'emblée

Le serveur **n'efface jamais les messages**.

Constat vérifié dans le code. `dataRetentionService.js` purge les statuts expirés (7 jours), l'historique d'appels (12 mois), le journal de connexions (90 jours), les jobs en échec, les appareils révoqués. La table `message`, elle, n'apparaît nulle part dans ces purges. Seuls les *fichiers* médias sont purgés, par `mediaRetention.js`.

Autrement dit, sur un téléphone neuf, l'inscrit qui se reconnecte récupérerait de toute façon ses conversations, ses messages et ses contacts en resynchronisant. Alors pourquoi sauvegarder ce que le serveur possède déjà ?

Parce que la resynchronisation se paie exactement dans la monnaie que la note protège. L'application pagine : `getMessages` descend 50 messages à la fois, et `loadOlderMessages` remonte l'historique par tranches de 30. Reconstruire trois ans de conversations de cette façon, ce sont des milliers d'allers-retours, notre bande passante sortante et le forfait data de l'inscrit — pour des octets qu'une archive de quelques mégaoctets aurait livrés en une seule fois.

La bonne façon de présenter la sauvegarde. Pour l'essentiel de son contenu, elle n'est pas un *filet de sécurité* mais une *économie* : elle évite de refaire par le réseau un travail déjà fait. C'est la même logique que l'exportation, appliquée à la base au lieu des médias. Elle redevient un vrai filet pour le noyau que le serveur ignore, décrit ci-dessous.

Trois natures de données, trois raisons différentes

Nature	Exemples	Motif
Le serveur l'a aussi	Messages, conversations, contacts, profil, réactions, listes de contacts, trajets	Économie
Seul l'appareil l'a	Traductions ML Kit (<code>translatedContent</code> , <code>sourceLang</code> , <code>translationState</code>); appels de plus de 12 mois; préférences d'affichage et de téléchargement	Filet
À ne pas restaurer	Chemins de fichiers locaux, file d'envoi, jetons de session, verrou biométrique	Nuisible

La deuxième ligne mérite un mot. Les colonnes de traduction portent en commentaire, dans `app_database.dart`, la mention explicite : « *ces trois colonnes ne voyagent jamais : elles ne sont ni envoyées au serveur ni reçues de lui* ». Un inscrit qui a fait traduire deux ans de conversations sur son appareil perdrait tout ce travail sans sauvegarde — et l'application relancerait l'identification de langue sur tout l'historique au premier lancement.

L'historique d'appels est le second cas net : le serveur le purge à 12 mois, l'appareil le garde. Passé un an, la sauvegarde est le seul endroit où ces appels existent encore.

Les préférences ne sont pas dans la base

C'est la correction qui justifiait cette section. Les réglages ne vivent pas dans Drift mais dans `SharedPreferences`, un magasin séparé qu'une sauvegarde de la seule base laisserait entièrement de côté : langue, thème, échelle de police, réduction des animations, téléchargement automatique, visibilité des médias, mode d'aperçu des notifications.

Toutes ne doivent pas voyager pour autant. Beaucoup sont déjà côté serveur et reviennent seules à la connexion ; d'autres sont propres au matériel.

Constat vérifié dans le code. `PrivacyPrefsService` ne fait que *mettre en cache* la réponse de `getPrivacyPrefs()` sous la clé `privacy_prefs_json_v1`. Les préférences de confidentialité, de notification et les plages « ne pas déranger » ont chacune leur contrôleur côté serveur. Les sauvegarder les dupliquerait, avec le risque qu'une valeur périmée écrase la valeur serveur au moment de la restauration.

Le détail de ce tri figure au chapitre 5, §5.2.

Quel poids, réellement

« **Quelques mégaoctets** » mérite d'être nuancé. Le poids de la sauvegarde n'est pas dicté par le nombre de messages mais par les **vignettes vidéo**. `mediaThumb` stocke une image JPEG encodée en base64 sur la ligne du message, de l'ordre de 15 Ko pièce. Cinq cents vidéos reçues, et les vignettes pèsent à elles seules davantage que tout le texte de l'historique.

Ordre de grandeur pour un compte chargé — 20 000 messages, 500 vidéos — une fois compressé :

Poste	Brut	Après gzip
Texte, métadonnées, index	~7 Mo	~2 Mo
Vignettes vidéo (<code>mediaThumb</code> , base64)	~7,5 Mo	~5,5 Mo
Total	~15 Mo	~7,5 Mo

La compression est efficace sur le texte, et rattrape sur les vignettes le tiers de surcoût que l'encodage base64 avait ajouté. L'estimation reste à confirmer par la mesure : `StorageInfoService` expose déjà `databaseBytes`, il suffit de l'instrumenter sur quelques comptes réels avant de figer les seuils d'alerte.

Pourquoi garder les vignettes malgré leur poids. Elles sont ce qui rend l'historique restauré immédiatement lisible. Les fichiers pleins arrivent en tâche de fond pendant plusieurs minutes ; sans les vignettes, l'inscrit contemplerait pendant tout ce temps des colonnes de rectangles gris à la place de ses souvenirs. Cinq mégaoctets pour que la restauration ait l'air réussie dès la première seconde, c'est bien acheté.

3.2 Où, et sous quelle forme

Dans le dossier Alanya/ du Drive de l'inscrit — visible, pas caché. Il voit ses sauvegardes, connaît leur poids, peut les effacer pour faire de la place. Le champ d'accès demandé à Google se limite aux fichiers créés par l'application : Alanya ne voit rien d'autre de son Drive.

L'archive est **chiffrée**, avec une clé dérivée du compte Alanya et fournie par le serveur à la connexion. L'inscrit n'a rien à retenir, et la restauration est transparente.

Ce n'est pas du chiffrement de bout en bout. La clé étant dérivée côté serveur, Alanya peut techniquement déchiffrer une sauvegarde. Google, lui, ne le peut pas. C'est un arbitrage assumé contre l'alternative — un mot de passe choisi par l'inscrit — dont le mode de panne (mot de passe oublié = sauvegarde perdue à jamais) est le premier motif de support sur ce type de fonction. Ce point doit figurer dans la politique de confidentialité ; le taire serait une faute.

3.3 Deux sauvegardes, jamais une seule

On conserve la sauvegarde courante **et** la précédente. La nouvelle n'écrase l'ancienne qu'une fois écrite, relue et vérifiée. Le coût est dérisoire — la base pèse quelques mégaoctets — et il ferme le scénario qui rend une sauvegarde inutile : une écriture interrompue qui détruit la dernière bonne copie.

3.4 Quand elle se déclenche

La fréquence est au choix de l'inscrit : *quotidienne, hebdomadaire, mensuelle, jamais*. C'est la convention que WhatsApp a installée et qu'il saura lire sans explication. Une sauvegarde manuelle reste possible à tout moment, et c'est elle qu'on met en avant juste avant un changement de téléphone.

Constat vérifié dans le code. La condition « seulement en Wi-Fi » est réalisable : `ConnectivityService` distingue déjà les réseaux non facturés (**wifi**, **ethernet**) des données mobiles.

3.5 Quand elle échoue

Quota Google plein, réseau coupé, autorisation retirée : les échecs passagers sont réessayés en silence. Ce n'est qu'après **plusieurs jours consécutifs sans aucune sauvegarde réussie** que l'application le signale visiblement, avec la cause et la marche à suivre. Alerter à chaque coupure réseau apprendrait à l'inscrit à ignorer nos alertes — et il les ignorerait aussi le jour où elles comptent.

3.6 L'inscrit sans compte Google

Tous n'en ont pas, et certains refuseront d'en connecter un. Ils ne doivent pas rester sans filet : l'application écrit alors la même archive dans le stockage du téléphone, sous `Download/Alanya/Sauvegardes/`, à charge pour l'inscrit de la recopier où il veut. Même format, même chiffrement, même restauration. Seule la destination change.

3.7 La restauration sur un nouveau téléphone

À la première connexion sur un appareil neuf, l'application cherche une sauvegarde et propose de la restaurer. Le déroulement est coupé en deux, parce que les deux moitiés n'ont pas du tout le même poids.

1. **La base, en étape bloquante.** Quelques mégaoctets, quelques secondes, un écran dédié. Aucune ambiguïté sur l'état des données : quand l'inscrit entre dans l'application, son historique est là.
2. **Les médias, en tâche de fond.** Les archives d'export trouvées dans le dossier Alanya/ sont rapatriées et redistribuées derrière, reprise après coupure comprise, avec un bandeau qui dit où ça en est. L'application est utilisable pendant ce temps.

Toujours prévoir la sortie. Une étape bloquante sans échappatoire enferme l'inscrit dont la sauvegarde est corrompue ou illisible : il ne peut plus entrer dans l'application du tout. L'écran de restauration doit donc toujours porter un « Ignorer et continuer sans restaurer », et l'opération doit rester relançable ensuite depuis les réglages.

3.8 Suppression du compte

Quand l'inscrit supprime son compte Alanya, l'écran de suppression lui **propose** d'effacer aussi sa sauvegarde, sans le lui imposer. Ce fichier est dans son espace personnel : le détruire d'autorité se défendrait mal, et certains voudront le garder pour revenir.

PARTIE II

Comment on le construit

CHAPITRE 4

Le format d'archive

4.1 Le manifeste

```
1 {
2   "format": "alanya-export/1",
3   "alanyaID": 241030112,
4   "generatedAt": "2026-08-30T10:24:07Z",
5   "period": { "from": "2026-03-01", "to": "2026-03-31" },
6   "filters": { "conversationID": 12, "kind": "all", "owner": "all" },
7   "retentionDaysKnown": 30,
8   "items": [
9     {
10      "msgID": 90412, "clientId": "c_8f21ab", "conversationID": 12,
11      "conversationName": "Ama Nguema", "senderID": 77, "senderName":
12        "Ama Nguema",
13      "isMine": false, "type": 1, "sentAt": "2026-03-14T09:12:44Z",
14      "path": "Alanya Images/2026-03-14_ama-nguema_0001.jpg",
15      "originalName": "IMG_2231.jpg", "bytes": 2841233,
16      "sha256": "9f2c...", "mediaUrl":
17        "/uploads/media/2026-03-14/media_90412_...jpg"
18    },
19    { "msgID": 90455, "sentAt": "2026-03-19T18:02:00Z", "reason":
20      "expired" },
21    { "msgID": 90460, "sentAt": "2026-03-21T07:40:11Z", "reason":
22      "neverDownloaded" }
23  ]
24 }
```

Trois champs méritent qu'on s'y arrête.

- `msgID` et `clientId` ensemble, jamais l'un sans l'autre. `msgID` vaut 0 tant que le serveur n'a pas confirmé le message; `clientId` est stable dès l'origine. La restauration se cale sur `clientId` et n'utilise `msgID` que comme confirmation.
- `sha256` rend la restauration **idempotente**. Rejouer une archive déjà restaurée ne duplique rien, et un fichier tronqué par une coupure est détecté au lieu d'être installé.
- `missing` avec son motif : c'est ce qui transforme une archive incomplète en archive honnête.

4.2 L'assemblage, et pourquoi il ne compresse pas

Détail qui change tout sur un export de plusieurs gigaoctets. Photos JPEG, vidéos MP4, vocaux M4A et PDF sont **déjà compressés**. Les repasser dans l'algorithme *deflate* du zip coûte beaucoup de temps processeur et de batterie pour un gain de taille voisin de zéro — parfois négatif. L'archive est donc écrite en mode *stocké* pour ces familles, et compressée uniquement pour le manifeste et l'index HTML. Le zip ne sert pas à réduire, il sert à **tenir ensemble**.

L'assemblage lit les fichiers un par un depuis leur emplacement d'origine et les écrit au fil de l'eau dans l'archive. Aucune copie intermédiaire : une mise en scène dans un dossier temporaire doublerait l'espace disque nécessaire, ce qui, sur un téléphone qu'on cherche précisément à soulager, serait une faute.

Point de vigilance. Vérifier l'espace libre *avant* de commencer, et refuser proprement si l'archive ne tient pas. Un export qui remplit le disque à 98 % laisse l'appareil dans un état pire que celui d'où l'on partait.

4.3 Reprise après interruption

L'export se découpe en trois phases dont la reprise n'a pas le même coût.

Phase	Ressource	Reprise
1. Récupération des manquants	réseau	Gratuite : les fichiers déjà descendus sont dans le cache, la phase les saute.
2. Assemblage du zip	disque et processeur	Recommencée d'un trait. Sans réseau, c'est rapide ; un zip ne se reprend pas au milieu.
3. Dépôt sur Drive	réseau	Native : l'API Drive gère les téléversements reprenables.

Les deux phases coûteuses sont donc reprenables, et la seule qui ne l'est pas ne coûte pas de réseau. C'est ce découpage qui permet de tenir la promesse « tâche de fond reprenable » sans construire de machinerie de reprise maison.

4.4 Le service de premier plan et sa limite de six heures

Un export long tourne dans un service de premier plan. Android encadre désormais ce type de service, et il faut le regarder en face avant de promettre quoi que ce soit.

Vérifié dans la documentation Android. Les services de premier plan de type `dataSync` — comme `mediaProcessing` — sont plafonnés à **six heures par tranche de 24 heures**, pour les applications ciblant l'API 35 ou plus. Au terme, le système appelle `Service.onTimeout()` et laisse quelques secondes pour appeler `stopSelf()` ; passé ce délai il lève `RemoteServiceException`, c'est-à-dire un plantage. Le compteur repart quand l'inscrit ramène l'application au premier plan. Alanya cible l'API 36 : elle est concernée.

Six heures, rapportées à ce qu'on fait, c'est immense. L'assemblage du zip est local au disque : des minutes. Le téléversement d'une sauvegarde, c'est huit mégaoctets. Seul le rapatriement de plusieurs gigaoctets d'archives sur un réseau lent pourrait en approcher — et il est déjà reprenable, donc une expiration ne fait que reporter la suite. Le plafond n'est pas une contrainte pour cette fonctionnalité ; c'en serait une pour un traitement d'un seul tenant, et c'est une raison de plus de ne pas en faire un.

Trois exigences suffisent, et aucune n'est coûteuse :

1. Déclarer `FOREGROUND_SERVICE_DATA_SYNC` et `android:foregroundServiceType="dataSync"` dans le manifeste — qui déclare aujourd'hui `phone_call`, `microphone`, `camera` et `location`, mais pas celui-ci.
2. Implémenter `Service.onTimeout()` : persister la progression, notifier, appeler `stopSelf()` **en quelques secondes**. Ne pas le faire est un plantage, pas un avertissement.
3. Reprendre au tick suivant, ce que le découpage en trois phases permet déjà.

Et cela se teste, sans attendre six heures.

```
1 adb shell am compat enable FGS_INTRODUCE_TIME_LIMITS
  com.alanya237.alanya
2 adb shell device-config put activity_manager
  data_sync_fgs_timeout_duration 30000
```

Le délai tombe à trente secondes. Le comportement d'expiration devient un test, au lieu d'un espoir.

4.5 Mesurer avant d'annoncer

La feuille d'export annonce un poids avant que l'inscrit ne s'engage. Ce chiffre ne peut pas venir de la base.

La colonne ment, et pas qu'un peu. `mediaSize` est nullable — « nul pour les médias envoyés avant que l'app ne relève la taille des images et vidéos ». Sommer la colonne fait donc compter **zéro octet** pour tout média ancien. Le même défaut touchait l'écran *Mes médias* : son total était sous-évalué, et surtout son tri « Plus lourds » reléguait les anciens en fin de liste — un tri censé faire remonter les gros fichiers cachait précisément les plus anciens.

La mesure se fait donc au disque. C'est bon marché : quelques centaines d'appels `stat`, de l'ordre de la milliseconde, et le contrôle d'existence des fichiers en coûtait déjà autant.

Correctif déjà appliqué. `ChatDao.setMediaSize(clientId, bytes)` et le rattrapage dans `_onRows` sont écrits et testés. Un `stat` donne l'existence et le poids d'un seul appel ; quand la colonne est nulle, la taille mesurée est réécrite en base. **L'écran répare la donnée en la lisant** : le total, le tri et les pastilles de poids deviennent justes définitivement, et l'export héritera d'une base déjà assainie.

Le même code sert les deux usages — l'affichage et l'estimation d'export — pour qu'ils ne puissent pas diverger.

CHAPITRE 5

La sauvegarde

5.1 Fabriquer l’instantané

1. `VACUUM INTO` vers un fichier temporaire. C’est la seule manière d’obtenir un instantané **cohérent** de SQLite sans arrêter l’application : copier `talky_chat.sqlite` pendant qu’une écriture est en cours produirait une base corrompue.
2. **Neutralisation** des colonnes d’état local, *sur la copie* et jamais sur la base vivante (§5.2), et purge des messages fantômes.
3. Ajout du `prefs.json` issu de la liste blanche des préférences.
4. Compression *gzip* — ici elle est payante, une base SQLite se comprime bien.
5. Chiffrement AES-GCM avec la clé du compte.
6. Téléversement, sous le nom `alanya-backup-<alanyaID>-<horodatage>.enc`.
7. Vérification de la taille et de l’empreinte relues, *puis seulement* suppression de l’avant-dernière.

Un second fichier, `alanya-backup-<alanyaID>-latest.json`, reste **en clair** : il porte la date, la taille, le nombre de messages et la version de schéma. L’écran de réglages peut ainsi afficher « dernière sauvegarde : hier, 21 h 04, 8,2 Mo » sans avoir à déchiffrer quoi que ce soit, ni même à demander la clé au serveur.

5.2 Le tri : ce qui monte, ce qui reste

5.2.1 Les colonnes à neutraliser

Douze tables composent la base. Aucune n’est exclue en bloc — ce sont des *colonnes*, au sein de `LocalMessages`, qu’il faut vider avant l’instantané. Toutes décrivent l’état de cet appareil-ci, et les restaurer telles quelles produirait des pannes silencieuses.

Colonne	Pourquoi elle ne doit pas être restaurée
<code>localMediaPath</code>	Chemin absolu vers <code>media_cache</code> sur l' <i>ancien</i> téléphone. Restauré tel quel, il fait croire à l'application que le fichier est là : la grille <i>Mes médias</i> affiche des éléments qui n'existent pas, et la bulle échoue à l'ouverture.
<code>pendingUploadPath</code>	Fichier en attente d'envoi, resté sur l'ancien appareil. L'outbox tenterait indéfiniment d'envoyer un fichier introuvable.
<code>syncPending, lastEmittedAt, retryCount, failureCode</code>	État de la file d'envoi. Une sauvegarde prise pendant une coupure réseau ressusciterait, des mois plus tard, des messages en attente que le serveur a peut-être déjà reçus.

Le cas `localMediaPath` n'est pas qu'un détail d'hygiène. Il faut le vider *et* laisser la restauration des archives d'export le renseigner à nouveau, fichier par fichier, à mesure qu'elle réinstalle les médias. C'est le point de jonction entre les deux mécanismes de ce dossier : la sauvegarde rétablit les messages, les archives rétablissent les fichiers, et cette colonne est la couture entre les deux.

Les messages fantômes se traitent au même moment : `ChatDao.purgeGhostMessages` existe déjà pour les optimistes anciens jamais confirmés. L'appeler avant l'instantané évite de sauvegarder des lignes que l'application effacerait de toute façon au premier lancement.

5.2.2 Les préférences : une liste blanche, jamais une liste noire

`SharedPreferences` contient une quarantaine de clés, dont des jetons d'accès. La sauvegarde ne doit embarquer que celles qu'on a explicitement désignées.

Ce sens-là et pas l'autre. Avec une liste noire, toute clé ajoutée plus tard entrerait dans la sauvegarde par défaut — y compris un jeton, si quelqu'un en introduit un sans penser à ce document. La liste blanche fait payer l'oubli du bon côté : une préférence nouvelle ne sera simplement pas restaurée, et personne n'y perdra sa sécurité.

Ce qui monte	Ce qui reste, et pourquoi
app_locale, theme_mode, app_font_scale, app_reduce_motion	*_access_token, *_refresh_token, *_api_base — jetons de session, sans valeur ailleurs et dangereux à recopier
auto_download_media, media_wifi_only, media_data_saver, media_visibility	privacy_prefs_json_v1 — simple cache du serveur, qui le renverra
notif_preview_mode	biometric_lock_enabled — lié au matériel de l'ancien appareil
call_ringtone_selected_id (l'identifiant, jamais le chemin)	call_ringtone_active_path, call_ringtone_native_name — chemins locaux vers des fichiers absents
	pending_*, notif_dedup_v1, ended_call_ids_v1, status_seen_ids, countries_cache_v1 — files et caches propres à l'appareil
	media_retention_days_learned — réappris au premier 410
	exported_media_msg_ids_v2 — journal d'export local, dénué de sens ailleurs

Les préférences retenues voyagent dans un `prefs.json` joint à l'instantané, à l'intérieur de l'archive chiffrée. Les sonneries de liste ne figurent nulle part : elles sont déjà synchronisées par compte côté serveur.

5.2.3 Le garde-fou : un test qui refuse les clés non classées

Une liste blanche écrite une fois se périme en silence. Six mois plus tard, quelqu'un ajoute une préférence, ne connaît pas ce document, et elle n'est simplement jamais restaurée — sans que rien ne le signale.

La parade idéale serait un registre `PrefKeys` référencé partout. Elle touche les trente-deux fichiers qui utilisent `SharedPreferences` : trop cher pour ce qu'elle rapporte ici.

La parade retenue tient en une quarantaine de lignes : **un test qui balaie les sources**. Il collecte les littéraux de clés déclarés dans `lib/`, et vérifie que chacun figure dans exactement un des trois ensembles.

```

1 lib/core/services/backup/backup_prefs_policy.dart
2   kBackedUpKeys      app_locale, theme_mode, app_font_scale,
3                       app_reduce_motion,
4                       auto_download_media, media_wifi_only,
5                       media_data_saver,
6                       media_visibility, notif_preview_mode,
7                       call_ringtone_selected_id
8   kServerOwnedKeys   privacy_prefs_json_v1, ...    -> le serveur les
9                       renverra
10  kDeviceLocalKeys    *_access_token, biometric_lock_enabled,
11                       pending_*, ...

```

Un test qui lit le code source, c’est inhabituel — et c’est le seul qui tienne. Un test qui interrogerait `SharedPreferences` à l’exécution ne verrait que les clés déjà écrites sur l’appareil de test, donc jamais celle qu’on vient d’ajouter. Seule l’inspection des sources voit une clé *déclarée*. Le fichier de classification devient du même coup la documentation de la règle, et l’échec du test nomme la clé fautive : la décision est forcée, pas suggérée.

5.3 La clé, et son numéro de version

Le serveur expose un point d’accès authentifié qui renvoie une clé dérivée d’un secret serveur et de l’`alanyaID`. Elle n’est jamais écrite en clair sur le téléphone : on la demande au moment de sauvegarder, on la demande au moment de restaurer, on l’oublie entre les deux.

C’est cette dérivation, et non le compte Google, qui rattache une sauvegarde à son propriétaire : **une archive du compte A posée dans le Drive de B reste illisible pour B**. C’est pourquoi le rattachement peut rester souple.

5.3.1 Le jour où le secret doit tourner

Sans numéro de version, la rotation est une catastrophe. Fuite, changement d’hébergeur, simple hygiène : un secret serveur finit toujours par devoir être remplacé. Si les archives ne portent aucune indication de la clé qui les a chiffrées, ce remplacement rend **toutes les sauvegardes existantes illisibles d’un coup**, sans recours ni avertissement. Posé maintenant, le correctif tient en un champ. Ajouté après coup, il faut composer avec un parc d’archives anonymes.

Chaque archive porte donc un en-tête **en clair**, devant le contenu chiffré :

```
1 ALNB | version de format | kid | algorithme | nonce | alanyaID |
   <contenu chiffre>
```

`kid` est le numéro de version de la clé. Il dit *comment* déchiffrer sans qu’il soit besoin de détenir la clé, ce qui permet de décider avant d’agir.

5.3.2 Deux points d’accès, une table qu’on n’efface jamais

Point d’accès	Rôle
GET /backup/key	Le <code>kid</code> courant et sa clé — pour écrire.
GET /backup/key/:kid	La clé d’une version donnée — pour relire une archive ancienne.

Côté base, une table `backup_key_secrets(kid, secret, created_at, retired_at)` dont **aucune ligne n’est jamais supprimée**. Retirer une version signifie uniquement *cesser d’écrire avec*, jamais *cesser de servir*. La clé effectivement utilisée est dérivée du secret de la version et de l’`alanyaID`, si bien qu’une même version produit une clé différente pour chaque compte.

La rotation se réduit alors à insérer une ligne et à la marquer courante. Les nouvelles sauvegardes emploient le nouveau secret, les anciennes restent lisibles. Aucune campagne de rechiffrement, aucune interruption, aucune archive perdue.

5.3.3 Deux compléments à poser en même temps

— **Le `kid` figure aussi dans le `latest.json` en clair.** L’écran de restauration peut ainsi annoncer « sauvegarde illisible, version de clé inconnue » *avant* de télécharger huit mégaoctets

pour rien.

- **Les appels à `/backup/key` sont journalisés et limités en débit.** Puisque le serveur peut déchiffrer, ce point d'accès est le joyau de la couronne. Une trace d'audit fait toute la différence entre « Alanya peut déchiffrer » — phrase inquiétante — et « Alanya peut déchiffrer, et chaque accès laisse une trace » — phrase défendable devant un inscrit comme devant un régulateur.

5.4 Rattachement : un Drive par compte Alanya

La sauvegarde est marquée de l'`alanyaID`, pas du compte Google. Se reconnecter depuis un autre compte Google fonctionne, du moment qu'on ouvre le même compte Alanya ; deux comptes Alanya sur un même téléphone gardent deux sauvegardes distinctes.

Pourquoi ne pas figer le couple des deux comptes. Figer punirait le cas le plus banal : on change de téléphone, et c'est souvent l'occasion d'un autre compte Google ; ou bien l'accès à la boîte Gmail est perdu. La sauvegarde deviendrait irrécupérable, ce qui vide la fonction de son sens. Et le risque que cette souplesse semble ouvrir — restaurer la sauvegarde d'autrui — est déjà fermé par le chiffrement. Figer n'ajouterait aucune sécurité, seulement un mode de panne.

5.5 Restaurer une base plus ancienne que l'application

La base locale porte une version de schéma (aujourd'hui 26). Deux cas, deux traitements :

- **Sauvegarde plus ancienne que l'application.** Cas normal. Les migrations Drift s'exécutent à l'ouverture, comme pour n'importe quelle mise à jour.
- **Sauvegarde plus récente que l'application.** L'inscrit a rétrogradé, ou installe une version ancienne. La restauration doit **refuser**, en invitant à mettre l'application à jour. Ouvrir une base au schéma futur, c'est la corrompre.

CHAPITRE 6

La restauration

6.1 Le vrai risque n'est pas celui qu'on croit

Une restauration écrit des milliers de lignes dans `localMessages`. Or l'application possède déjà une machinerie qui écrit dans cette même table, à partir du serveur, et qui part **toute seule**.

Ce que ferait la conception naïve. `bootstrapEmptyConversationMessages()` se déclenche sans qu'on le lui demande, et sa condition d'entrée est exactement l'état d'un téléphone neuf. Son commentaire le dit : « *précharge les messages des conversations qui ont un aperçu serveur mais aucun message local confirmé (install fraîche, logout...)* ». Sans précaution, la connexion sur le nouveau téléphone déclencherait donc le rapatriement de tout l'historique par le réseau — pendant que la restauration écrit les mêmes lignes sur la même clé primaire. Il y aurait course d'écriture ; mais surtout, **la dépense réseau que la sauvegarde existait pour éviter aurait lieu quand même**. On aurait payé la sauvegarde pour n'économiser rien.

6.2 Deux bonnes nouvelles dans le code existant

Constat vérifié dans le code. Les curseurs sont déjà justes. Le bootstrap saute toute conversation qui possède déjà un message serveur : `if (await _dao.maxServerMsgId(c.conversID) > 0) continue; (conversation_sync.dart)`. Une fois la base restaurée, il ne cible plus rien et se tait de lui-même. Et `syncGlobalDelta` lit `_dao.conversationCursors()`, donc la synchronisation delta repart naturellement de la crête restaurée. **Aucune plomberie de curseur n'est à construire.**

Constat vérifié dans le code. La séquence de démarrage est linéaire et le socket arrive en dernier. Dans `main.dart`, l'enchaînement après authentification est ordonné, et le socket n'est connecté qu'après les `bind` — un commentaire du fichier explique pourquoi. Chaque chemin de synchronisation est par ailleurs gardé par `if (_myId == 0) return;`, et `_myId` n'est posé que par `ChatRepository.bind()`.

6.3 Quand demande-t-on le compte Google ?

Proposer une restauration suppose de savoir qu'une sauvegarde existe. Chercher dans Drive imposerait de connecter Google *avant* que l'inscrit soit entré dans l'application — au tout premier démarrage, sur un téléphone qu'il vient d'allumer. C'est le pire moment pour réclamer un compte tiers, et on le réclamerait aussi à ceux qui n'ont jamais rien sauvegardé.

La réponse est de ne pas interroger Drive du tout à ce stade : le serveur peut le savoir à sa place.

À chaque sauvegarde réussie, l'application déclare au serveur de simples métadonnées — date, taille, `kid`, et l'adresse Google **masquée**. Aucun contenu, jamais. À la connexion sur un appareil neuf, le serveur les rend, et l'application sait qu'une sauvegarde existe sans avoir touché à Google.

```

1  identifiants verifies
2  -> le serveur annonce : sauvegarde 29/08, 8,2 Mo
3      compte a...@gmail.com
4  -> "Restaurer" / "Ignorer"      <- Google demande ici
5  -> connexion Google + lecture Drive
6  -> restauration (bloquante)
7  -> chatProvider.bind(myId)
8  -> statusProvider.bind(myId)
9  -> connexion du socket

```

Trois gains, et le troisième est le moins évident :

- Un inscrit véritablement nouveau **n'est jamais sollicité** pour un compte Google.
- L'écran ne pose plus une question spéculative ; il annonce un fait daté et chiffré, ce qui rend le choix « Restaurer / Ignorer » réellement éclairé.
- Le cas « mauvais compte Google » cesse d'être une impasse. Sans cette métadonnée, l'application ne sait dire que « aucune sauvegarde trouvée » — indiscernable d'une absence réelle. Avec elle : « une sauvegarde existe, connectez-vous avec a...@gmail.com ».

L'adresse est stockée masquée, jamais en clair. C'est une donnée personnelle dont nous n'avons aucun besoin complet : la première lettre et le domaine suffisent à guider l'inscrit vers le bon compte. En conserver davantage serait accumuler du risque pour rien.

Il faut enfin garder une entrée manuelle « restaurer une sauvegarde » dans les réglages : pour les sauvegardes antérieures à cette métadonnée, et pour le cas où le serveur l'aurait perdue.

6.4 La solution : un ordre, pas un verrou

```

1  identifiants verifies
2  -> ecran de restauration (bloquant)      <- le point d'insertion
3  -> chatProvider.bind(myId)
4  -> statusProvider.bind(myId)
5  -> connexion du socket

```

Rien ne court encore, donc rien ne peut se percuter. Il n'y a ni verrou à poser, ni file d'attente à gérer, ni état concurrent à arbitrer.

Une décision produit qui paie une dette technique. C'est le choix d'une *restauration bloquante* qui rend cette solution possible. Une restauration en arrière-plan aurait imposé un verrou, une reprise de curseur et un arbitrage entre deux écrivains. La contrepartie — quelques secondes d'attente au premier démarrage — achète la disparition complète d'une classe de bugs.

6.5 L'état de restauration, et pourquoi il est indispensable

C'est la seule pièce réellement nouvelle : un état persisté hors de la base, puisque la base est justement ce qu'on est en train de remplacer.

État	Signification
inconnu	On n'a pas encore cherché de sauvegarde.
proposée	Une sauvegarde existe, l'inscrit n'a pas tranché.
en cours	Écriture commencée. La base est dans un état intermédiaire.
faite	Terminée et vérifiée.
ignorée	L'inscrit a refusé. On ne repropose plus au démarrage.

Une demi-restauration est pire que pas de restauration. Si l'application est tuée au milieu de l'écriture, la base contient des messages pour *certaines* conversations seulement. Au redémarrage, le bootstrap les considère comme complètes — puisque `maxServerMsgId > 0` — et les saute définitivement. **Les trous deviennent permanents, et silencieux.** C'est précisément parce que la logique de curseur est correcte qu'un état intermédiaire est toxique. La règle est donc sans nuance : trouver l'état **en cours** au démarrage impose de vider la base et de repropose la restauration depuis le début.

6.6 Les cinq répétitions à écrire avant le code

Une restauration ne se vérifie pas en la relisant. Ces cinq scénarios doivent être des tests, et ils doivent exister avant l'implémentation qu'ils décrivent.

Scénario	Comportement attendu
Archive tronquée	Détectée à la vérification d'intégrité. La base n'est pas touchée, l'inscrit peut réessayer ou ignorer.
Schéma plus récent que l'app	Refus explicite, invitation à mettre à jour. Jamais d'ouverture d'une base au schéma futur.
kid inconnu	Message clair avant tout téléchargement, grâce au <code>latest.json</code> (§5.3).
Coupure en pleine écriture	Au redémarrage, état en cours détecté : purge et nouvelle proposition.
Aucune sauvegarde trouvée	Distinguer « ce compte Google n'en contient pas » de « erreur réseau ». Proposer d'essayer un autre compte Google dans le premier cas.

Toujours une porte de sortie. Chacun de ces écrans doit porter « Ignorer et continuer sans restaurer », et l'opération doit rester relançable depuis les réglages. Une étape bloquante sans échappatoire enferme dehors l'inscrit dont la sauvegarde est illisible.

6.6.1 Ces répétitions imposent une contrainte d'architecture

Écrire ces cinq scénarios est facile ; les rendre *jouables* ne l'est que si on le décide avant de coder.

La règle qui rend le plan de test possible. `RestoreService` reçoit son `BackupTarget` et son fichier source **en paramètres**. Il ne va jamais les chercher dans un singleton, ni ouvrir Drive lui-même. Sans cette règle, les cinq scénarios ne seraient rejouables qu'à la main sur un vrai téléphone avec un vrai compte Google — autant dire jamais.

Avec elle, un `FakeBackupTarget` sert des archives fabriquées, et chaque scénario tient en quelques lignes :

Fixture	Fabrication
Archive tronquée	Une archive valide, coupée aux trois quarts.
kid inconnu	En-tête valide, kid inexistant côté serveur.
Schéma futur	Base au <code>schemaVersion</code> 99.
Mauvais compte	Archive d'un autre <code>alanyaID</code> , donc indéchiffrable.
Cible vide	<code>FakeBackupTarget</code> qui renvoie une liste vide.
Coupure	Interruption injectée au milieu de l'écriture ; on vérifie qu'un redémarrage trouve l'état <code>en cours</code> et purge.

Constat vérifié dans le code. Le socle de test existe déjà. `AppDatabase.forTesting(NativeDatabase.memory())` est utilisé par la suite actuelle : restaurer dans une base en mémoire et vérifier son contenu est directement à portée, sans émulateur ni appareil.

CHAPITRE 7

Ce qui reste à lever

7.1 L'inconnue qui conditionne l'architecture

Toute la restauration repose sur une hypothèse non vérifiée : sur un téléphone neuf, l'application retrouve dans le Drive de l'inscrit les fichiers qu'elle y avait déposés depuis l'ancien. Le champ d'accès retenu limite l'application aux fichiers qu'elle a créés — reste à savoir si ce droit est rattaché à **l'application** (identifiant client OAuth) ou à **l'installation**.

Si le droit suit... Conséquence

l'application	La sauvegarde est visible sur le nouveau téléphone. La conception fonctionne telle qu'écrite.
l'installation	L'application est aveugle à une archive pourtant présente. La restauration automatique disparaît.

Les trois issues en cas de réponse défavorable, et aucune n'est indolore : basculer sur le dossier caché (on perd le dossier visible que l'inscrit peut gérer, et la vérification Google s'alourdit), faire choisir le fichier à la main par le sélecteur Google (la restauration automatique disparaît du parcours, au pire moment — le premier démarrage d'un téléphone neuf), ou demander l'accès complet au Drive (audit annuel payant, hors de question).

7.1.1 Isoler d'abord

Toute la conception ne touche Drive que par quatre gestes : lister, écrire, lire, supprimer. Derrière une interface `BackupTarget`, la réponse au test ne coûte plus qu'une classe :

```
1 abstract class BackupTarget {
2     Future<List<RemoteArchive>> list();
3     Future<void> write(File local, String name);
4     Future<File> read(String id, File into);
5     Future<void> delete(String id);
6 }
7     DriveVisibleFolderTarget    <- choix par défaut
8     DriveAppDataTarget          <- repli si le test echoue
9     LocalFolderTarget           <- deja requis (sans compte Google)
```

Cette abstraction ne coûte rien de plus. La décision prise sur les inscrits sans compte Google imposait déjà `LocalFolderTarget`. L'interface devait donc exister de toute façon : elle protège du risque Drive par pure retombée.

7.1.2 Tester ensuite — et deux variantes, jamais une seule

1. Déclarer l'identifiant client OAuth dans la console Google.
2. Depuis un simple script — Flutter n'est pas nécessaire — s'authentifier avec un compte de test et créer `Alanya/test.txt`.
3. Rejouer l'accès de deux façons :
 - **Réinstallation simple** : repartir d'un jeton vierge sans rien révoquer. C'est le cas réel le plus fréquent.
 - **Révocation puis ré-autorisation** : l'inscrit a retiré l'accès depuis son compte Google, ou trop de temps a passé.
4. Redemander la liste dans chacun des deux cas.

Ne pas se contenter de la première variante. Les deux peuvent parfaitement donner des réponses différentes : une révocation est susceptible de jeter les droits accordés fichier par fichier, là où une réinstallation les conserverait. N'éprouver que la réinstallation donnerait une fausse assurance.

Coût : une demi-journée. À faire **avant** la première ligne du reste.

7.2 Les points désormais tranchés

Cette section listait cinq questions ouvertes. Elles ont toutes reçu une réponse, consignée au fil du document ; on n'en garde ici que la table de renvoi, pour que rien ne se perde.

Sujet	Réponse	Où
Service de premier plan	Type <code>dataSync</code> déclaré, <code>onTimeout()</code> implémenté, travail reprenable. Le plafond de six heures n'est pas une contrainte pour ce que nous faisons.	§4.4
Poids annoncé avant export	Mesure au disque et rattrapage en base. Correctif déjà appliqué au code de « Mes médias ».	§4.5
Moment de la connexion Google	Le serveur porte la métadonnée de sauvegarde : Google n'est demandé qu'après un « Restaurer » explicite.	§6.3
Plan de test	Six fixtures, et la règle d'injection qui les rend jouables.	§6
Garde-fou de la liste blanche	Un test qui balaie les sources et refuse toute clé non classée.	§5.2

Le seul point encore ouvert de ce chapitre. Reste la vérification que l'exportation hérite bien de l'exclusion des médias à vue unique. `watchLocalMedia` les écarte déjà côté destinataire et les conserve côté expéditeur ; il faut confirmer que ce comportement est celui qu'on veut dans une archive en clair. C'est une relecture de dix minutes, pas une décision d'architecture.

CHAPITRE 8

Impact sur le code

8.1 Ce qui existe et se réutilise

Existant	Rôle dans cette conception
ChatDao.watchLocalMedia	Fournit déjà le périmètre exact à exporter, filtres compris.
AlanyaMediaExportService	Arborescence des dossiers, album galerie, journal des identifiants déjà exportés.
MediaExpiryPolicy	Distingue « récupérable » de « purgé » sans tenter la requête.
ConnectivityService	Condition Wi-Fi de la sauvegarde (<code>wifi</code> , <code>ethernet</code>).
StorageInfoService	Espace libre avant export, poids de la base pour la sauvegarde.
flutter_local_notifications	Notification de progression.
CallMediaForegroundService	Motif de service de premier plan déjà maîtrisé, à reprendre pour l'export long.

8.2 Ce qu'il faut ajouter

Dépendance	Pourquoi
google_sign_in	Connexion au compte Google.
googleapis (Drive v3)	Dépôt, listage, téléversement reprenable.
archive	Écriture du zip au fil de l'eau, mode stocké.
cryptography	AES-GCM.
workmanager	Planification périodique de la sauvegarde.

Démarche Google à ne pas découvrir la veille de la publication. Le champ d'accès retenu se limite aux fichiers créés par l'application, c'est le moins exigeant de ceux qui conviennent — nettement plus léger que l'accès complet au Drive, qui impose un audit de sécurité annuel et payant. Il n'en reste pas moins qu'un écran de consentement OAuth doit être publié et que l'application doit être vérifiée par Google avant mise en production. À vérifier contre la politique Google en vigueur au moment du développement, et à lancer **tôt** : le délai n'est pas maîtrisé par l'équipe.

8.3 Fichiers à créer

```
1 lib/core/services/export/  
2   media_export_job.dart      phases, progression, reprise  
3   export_manifest.dart      ecriture et relecture du manifeste
```

```

4   export_zip_builder.dart      assemblage au fil de l'eau
5   export_index_html.dart      planche-contact
6
7   lib/core/services/backup/
8     backup_service.dart        orchestration
9     backup_crypto.dart         AES-GCM, en-tete ALNB, kid
10    backup_scheduler.dart       frequence, conditions, reessais
11    backup_snapshot.dart        VACUUM INTO, neutralisation, prefs.json
12    backup_target.dart          interface : list / write / read /
                                delete
13    drive_visible_folder_target.dart  choix par defaut
14    drive_appdata_target.dart      repli si le test Drive echoue
15    local_folder_target.dart       inscrit sans compte Google
16    restore_service.dart          base bloquante, medias en fond
17    restore_state.dart            inconnu / proposee / en cours / faite
                                / ignoree
18
19   lib/screens/profile/
20     backup_screen.dart           reglages et etat
21     restore_screen.dart          etape bloquante, avec sortie
22     my_media_screen.dart         + bouton et feuille d'export
23
24   lib/main.dart                 insertion de l'etape de restauration
25                                AVANT chatProvider.bind(myId)

```

8.4 Côté serveur

Deux points d'accès et une table, tous décrits au §5.3 : **GET** /backup/key pour écrire, **GET** /backup/key/:kid pour relire une archive ancienne, et backup_key_secrets dont aucune ligne n'est jamais supprimée. Les deux points d'accès sont authentifiés comme le reste de l'API, journalisés et limités en débit.

La rétention, elle, n'appelle aucun développement : elle est déjà annoncée par le champ retentionDays des réponses 410.

CHAPITRE 9

Décisions arrêtées

Sujet	Décision
Bouton Exporter	Archive zip assemblée localement, remise au partage système, à Téléchargements, ou déposée sur Drive.
Contenu exporté	Médias de la période, plus <code>manifest.json</code> et <code>index.html</code> . Pas les messages texte — v2 possible.
Éléments manquants	Récupérables et purgés distingués ; téléchargement proposé, chiffré d'avance, non coché par défaut.
Contenu sauvegardé	La base, moins les colonnes d'état local, plus une liste blanche de préférences. Médias exclus.
Tri des préférences	Liste blanche explicite. Jamais une liste noire.
Clé de sauvegarde	Numérotée (<code>kid</code>) dans un en-tête en clair. Table de secrets dont aucune ligne n'est supprimée. Accès journalisé.
Ordre au démarrage	La restauration s'insère <i>avant</i> <code>chatProvider.bind(myId)</code> . Un ordre, pas un verrou.
État de restauration	Persisté hors base. en cours au démarrage $\Rightarrow$ purge et nouvelle proposition.
Accès au stockage	Interface <code>BackupTarget</code> , trois implémentations. Le résultat du test Drive ne coûte qu'une classe.
Emplacement	Dossier <code>Alanya</code> visible du Drive de l'inscrit.
Chiffrement sauvegarde	AES-GCM, clé dérivée du compte côté serveur. Pas de bout en bout, et on le dit.
Chiffrement archive export	Aucun. Elle doit s'ouvrir sans <code>Alanya</code> . L'écran avertit.
Rattachement	Un Drive par compte <code>Alanya</code> . Compte Google non figé.
Fréquence	Au choix : quotidienne, hebdomadaire, mensuelle, jamais. Manuelle toujours possible.
Versions conservées	Deux : la courante et la précédente. Écrasement seulement après vérification.
Échecs	Réessai silencieux, alerte visible après plusieurs jours sans succès.
Sans compte Google	Même archive écrite dans <code>Download/Alanya/Sauvegardes/</code> .
Restauration	Base en étape bloquante avec sortie possible ; médias en tâche de fond reprenable.
Suppression de compte	Effacement de la sauvegarde proposé, jamais imposé.
Badge de rétention	Valeur apprise du serveur, formulée par contraste : sans limite sur l'appareil, 30 jours sur les serveurs.
Plateforme	Android d'abord. Abstraction de destination prête pour iCloud.

9.1 Hors périmètre de ce lot

- Le menu **Plus** et la carte « **Il y a un an, ce jour-là** » de la maquette : fonction d’engagement, conception séparée.
- **iOS et iCloud** : l’abstraction de destination est prévue, l’implémentation ne l’est pas.
- Le **transfert direct d’appareil à appareil** en Wi-Fi local. C’est pourtant la réponse la plus fidèle à l’esprit de la note — aucun octet ni chez nous ni chez Google. À reconsidérer si le dépôt d’archives sur Drive se révèle trop peu utilisé.
- L’export du **fil de discussion lisible** en HTML.

9.2 Dépendances : état réel

Cette section listait quatre préalables. Vérification faite dans le code, deux étaient déjà levés.

Constat vérifié dans le code. `evictIfNeeded()` et le plafond de 500 Mo ont été **retirés** de `media_cache_service.dart`. Le commentaire du fichier porte le raisonnement : évincé côté appareil *et* tombé côté serveur = perdu définitivement ; les deux mécanismes étaient anodins séparément et destructeurs ensemble. La promesse « sans limite sur l’appareil » est donc tenue.

Constat vérifié dans le code. Le `410 MEDIA_EXPIRED` est **géré**. `MediaExpiryPolicy.noteResponse()` intercepte la réponse dans le chemin de téléchargement et mémorise la rétention annoncée ; `isExpired()` évite même d’émettre la requête. La distinction « récupérable » / « purgé » de la feuille d’export est donc réalisable dès maintenant.

Il reste deux points, et un seul concerne le code :

1. **Le test de visibilité Drive** (§7, deux variantes, une demi-journée). Il ne bloque que le choix entre `DriveVisibleFolderTarget` et `DriveAppDataTarget` — une classe derrière l’interface `BackupTarget`. Tout le reste peut être écrit sans attendre sa réponse.
2. **L’écran de consentement OAuth publié et l’application vérifiée par Google.** Ce n’est pas un préalable au code mais au *déploiement*, et le délai n’est pas maîtrisé par l’équipe : à lancer le premier jour, en parallèle du développement.

Ce qui n’est plus une inconnue. La course entre restauration et synchronisation s’est révélée être un simple problème d’ordre : les curseurs existants sont déjà justes et la séquence de démarrage est linéaire (chapitre 6).